Informe de seguridad y calidad en el desarrollo de software

Caso SmartLogix: plataforma eCommerce de perfumes en microservicios

Asignatura: ISY1102 - Seguridad y calidad en el desarrollo de software

Evaluación: Evaluación Parcial Nº3 (encargo)

Carrera: [completar]

Sección: 004V

Docente: Manuel Reyes Ferrada

Fecha: 14 de octubre de 2026

Integrantes

Nicolás Castillo

Benjamín Catalán

Benjamín Quiroz

Índice

1. Introducción 4
1.1 Contexto del software evaluado 4
1.2 Propósito del plan y de las pruebas 4
1.3 Origen de los requisitos 4
1.4 Ajustes a la Evaluación Parcial 1 4
1.5 Estructura del informe 5
2. Criterios de calidad, seguridad y cumplimiento normativo 5
2.1 Calidad del software (ISO/IEC 25010) 5
2.2 Seguridad 5
2.3 Cumplimiento normativo 6
3. Plan de pruebas 6
3.1 Alcance y objetivos 6
3.2 Estrategia de pruebas 7
3.3 Tipos de prueba y justificación 7
3.4 Criterios de aceptación 8
3.5 Herramientas y justificación 8
3.6 Recursos (humanos, técnicos y de entorno) 9
3.7 Cronograma 9
4. Diseño de casos de prueba 10
4.1 Matriz de trazabilidad requisito → caso → incidente 11
5. Ejecución de pruebas 13
5.1 Resumen de ejecución 13
5.2 Detalle de ejecución de casos 13
5.3 Incidentes identificados y criticidad 20
6. Análisis ético y normativo de los resultados 23
6.1 Análisis ético 23

6.2 Análisis normativo 23
7. Conclusiones y recomendaciones 24
Anexo A. Reproducibilidad 24
Anexo B. Cambios realizados al código 25

Clic derecho sobre el índice y 'Actualizar campos' para mostrar la numeración.

1. Introducción

1.1 Contexto del software evaluado

SmartLogix es un sistema de comercio electrónico de perfumes desarrollado con arquitectura de microservicios. Está formado por un frontend en React 18 y Vite, un API Gateway (BFF) con autenticación JWT sobre Spring Security, y tres microservicios independientes en Spring Boot 3.2 y Java 21: Inventory Service (catálogo y stock), Order Service (pedidos con el patrón Factory Method para los tipos de orden WEB y MARKETPLACE) y Shipping Service (envíos con Circuit Breaker de Resilience4j). Cada servicio usa su propia base de datos H2 en memoria.

1.2 Propósito del plan y de las pruebas

El plan busca comprobar que el sistema cumple los requisitos funcionales, de seguridad, de rendimiento y de mantenibilidad que se le exigen, y que lo hace de forma reproducible. Un eCommerce gestiona credenciales, datos personales (nombre, correo y dirección de envío) y pedidos, por lo que una falla de seguridad tiene consecuencias directas sobre los usuarios y sobre el cumplimiento de la Ley 19.628 y de su reforma.

1.3 Origen de los requisitos

Limitación declarada. El caso no dispone de un documento de especificación de requisitos (ERS) con códigos propios. Por ello, los requisitos de este informe se derivan de tres fuentes, que se indican en la sección 2: los requisitos y criterios del informe de la Evaluación Parcial 1, el código fuente del proyecto y la normativa aplicable. Los códigos REQ-SEG-01, REQ-FUNC-02, REQ-FUNC-03, REQ-RES-01 y REQ-VAL-01 provienen de la Ev1; el resto se añadió en esta evaluación a partir del código y de la normativa. La cobertura del 100 % se entiende respecto de este conjunto de requisitos, no de un ERS oficial.

1.4 Ajustes a la Evaluación Parcial 1

No se dispone de la retroalimentación escrita del docente a la Ev1, por lo que los ajustes se basan en la verificación contra el código y la ejecución. Se corrigieron las siguientes afirmaciones: la cobertura de la Ev1 (87,4 % backend y 84,1 % frontend) no se reproduce; el reporte de cobertura del frontend registra 68,52 % de statements; las respuestas sin token eran HTTP 403, no 401; el tipo de orden inválido respondía 404, no 500; y el rango de precio 50-150 del caso CP-06 no aplica porque los precios se almacenan en unidades enteras.

1.5 Estructura del informe

La sección 2 define los criterios de calidad, seguridad y cumplimiento. La sección 3 presenta el plan de pruebas. La sección 4 detalla los casos de prueba y la matriz de trazabilidad. La sección 5 registra la ejecución, los incidentes y su criticidad. Las secciones 6 y 7 contienen el análisis ético y normativo y las conclusiones.

2. Criterios de calidad, seguridad y cumplimiento normativo

2.1 Calidad del software (ISO/IEC 25010)

- **Funcionalidad:** los pedidos, descuentos, envíos y búsquedas deben producir los resultados especificados (REQ-FUNC-02, REQ-FUNC-03, REQ-RES-01).
- **Rendimiento:** lecturas con p95 inferior a 500 ms y escrituras con p95 inferior a 1000 ms en condiciones de uso normal (REQ-NFR-01).
- **Fiabilidad:** el Circuit Breaker debe activar el fallback ante fallo del carrier sin colapsar el servicio (REQ-RES-01).
- **Mantenibilidad:** cobertura de líneas JaCoCo de al menos 60 % en cada servicio y gate de build activo (REQ-CAL-01).
- **Usabilidad y compatibilidad:** se consideran en el plan (pruebas de usabilidad y navegadores Chrome, Firefox y Safari), pero no se ejecutaron en esta evaluación; se declara como limitación en la sección 3.3.

2.2 Seguridad

- **Autenticación y control de acceso:** los endpoints distintos de /api/auth/** requieren JWT; las credenciales inválidas se rechazan con 401 (REQ-SEG-01).
- **Protección de credenciales:** las contraseñas se almacenan como hash BCrypt (REQ-SEG-02).
- **Gestión de secretos:** la clave de firma JWT se obtiene de una variable de entorno y no se versiona (REQ-SEG-05).
- **Manejo de errores:** las respuestas de error indican el código correcto y no incluyen trazas ni mensajes del framework (REQ-SEG-03).
- **Integridad de entradas:** validación con Jakarta Validation y rechazo con HTTP 400 (REQ-VAL-01).
- **Configuración de red:** CORS limitado a http://localhost:5173 y http://localhost:3000 (REQ-SEG-04).

- **Dependencias:** sin vulnerabilidades críticas ni altas en dependencias; el estado real se reporta en INC-09 (REQ-SEG-06).

2.3 Cumplimiento normativo

Ley 19.628 y su reforma (Ley 21.719). La Ley 19.628 rige actualmente. La Ley 21.719 la reforma integralmente y, según la Biblioteca del Congreso Nacional, entra en vigencia el 1 de diciembre de 2026. El 1 de septiembre de 2026 el Gobierno propuso postergar esa fecha al 1 de diciembre de 2027. Este informe no puede confirmar el estado de esa propuesta; debe verificarse en bcn.cl antes de la entrega. Las obligaciones que se analizan en la sección 6 son las de la ley reformada, porque son las que el sistema deberá cumplir en el mediano plazo.

- **Principio de seguridad (art. 3 f) y medidas de seguridad (art. 14 quinquies):** cifrado o seudonimización, confidencialidad, integridad y resiliencia, y verificación periódica de la eficacia de las medidas.
- **Reporte de vulneraciones (art. 14 sexies):** comunicación a la Agencia sin dilaciones indebidas cuando exista riesgo razonable para los titulares.
- **Derechos del titular (arts. 4 y 9):** acceso, rectificación, supresión, oposición, portabilidad y bloqueo.
- **OWASP Top 10 (2021):** A01 control de acceso, A02 fallas criptográficas, A05 configuración incorrecta, A06 componentes vulnerables, A07 fallas de autenticación.
- **Estándares técnicos:** Java 21, Spring Boot 3.2, React 18, OpenAPI 3.0 y convenciones REST.

3. Plan de pruebas

3.1 Alcance y objetivos

El alcance comprende los cuatro servicios backend (BFF, inventario, pedidos y envíos), el frontend y el despliegue en contenedores. Quedan fuera la infraestructura de nube, la carga concurrente a escala de producción y las pruebas en dispositivos móviles.

- O1. Verificar los requisitos funcionales y de seguridad mediante casos ejecutados sobre el sistema real.
- O2. Medir la línea base del código original y el efecto de las mitigaciones con el mismo ejecutor.
- O3. Identificar y clasificar incidentes con criticidad y estado de tratamiento.
- O4. Dejar el sistema desplegable y las pruebas reproducibles con un único comando.

3.2 Estrategia de pruebas

El enfoque es mixto. Las pruebas unitarias y de integración son automatizadas (JUnit 5, Mockito y Vitest). Las pruebas funcionales y de seguridad se ejecutan contra los servicios en ejecución mediante un script reproducible (docs/pruebas/ejecutar_casos.py). El análisis estático y la auditoría de dependencias son automatizados. La inspección de código se usa cuando la propiedad a verificar no es observable desde el exterior (almacenamiento de contraseñas y configuración de secretos).

Etapas: (1) línea base con pruebas unitarias y cobertura; (2) análisis estático y SCA; (3) ejecución de casos sobre el código original; (4) mitigación de incidentes; (5) regresión sobre el mismo ejecutor en contenedores Docker; (6) documentación de resultados.

Priorización: crítica (autenticación y secretos), alta (pedidos, envíos y validación), media (búsqueda, rendimiento y fallback del Circuit Breaker), baja (cumplimiento documental).

Criterios de entrada: build compilado sin errores, servicios desplegados en sus puertos y bases H2 inicializadas. Criterios de salida: todos los casos críticos y altos aprobados, cobertura de líneas por servicio >= 60 % y ningún incidente crítico abierto sin tratamiento documentado.

3.3 Tipos de prueba y justificación

Tipo	Propósito en el aseguramiento de la calidad	Aplicación en SmartLogix	Estado
Funcional (caja negra)	Verificar que cada función produce el resultado especificado.	CP-03, CP-04, CP-06	Ejecutada
Unitaria (caja blanca)	Verificar unidades aisladas y su lógica interna.	112 pruebas JUnit y 55 Vitest	Ejecutada
Integración	Verificar la comunicación entre BFF y servicios.	CP-08 y pruebas de proxy	Ejecutada
Seguridad	Verificar autenticación, autorización, secretos, errores y CORS.	CP-01, CP-02, CP-07, CP-10 a CP-13	Ejecutada
Resiliencia	Verificar la recuperación ante fallos de dependencias.	CP-05 (Circuit Breaker)	Ejecutada
Rendimiento (no funcional)	Medir tiempos frente a los umbrales definidos.	CP-09 (20 muestras por operación, sin carga concurrente)	Ejecutada con limitaciones
Mantenibilidad (cobertura)	Medir la proporción de código verificado por pruebas.	CP-14 (JaCoCo)	Ejecutada
Análisis estático	Detectar defectos y vulnerabilidades sin ejecutar el código.	CP-15 (SpotBugs + FindSecBugs), ESLint	Ejecutada
Composición de software (SCA)	Detectar vulnerabilidades en dependencias de terceros.	CP-16 (npm audit)	Ejecutada
Despliegue	Verificar que el sistema se despliega de forma reproducible.	CP-17 (Docker, compose)	Ejecutada

Usabilidad	Evaluar la experiencia de navegación y el feedback visual.	Solo capturas de inicio (E07)	No ejecutada en profundidad
Compatibilidad de navegadores	Verificar el funcionamiento en Chrome, Firefox y Safari.	No realizada (solo Chrome headless)	No ejecutada

3.4 Criterios de aceptación

- **Funcional:** el resultado observado coincide con el esperado (estado, código HTTP y campos indicados).
- **No funcional:** p95 de lectura < 500 ms y de escritura < 1000 ms; cobertura de líneas >= 60 %.
- **Seguridad:** sin token o con credenciales inválidas, respuesta 401; sin exposición de trazas ni mensajes internos; secreto no literal en el repositorio.
- **Reglas de negocio:** descuento WEB 5 % (total 190,00) y MARKETPLACE 10 % (total 180,00); envío STANDARD o EXPRESS según el tipo.
- **Análisis estático:** cero hallazgos de prioridad 1 sin tratamiento documentado.

3.5 Herramientas y justificación

Herramienta	Tipo	Justificación
JUnit 5 y Mockito	Unitarias e integración (backend)	Estándar de Spring Boot; permite aislar dependencias.
Spring MockMvc (standalone)	Controladores y handlers	Verifica códigos HTTP y cuerpos de error sin levantar la aplicación completa.
JaCoCo 0.8.15	Cobertura (backend)	Gate de cobertura de líneas en el build (REQ-CAL-01).
Vitest 1.6 y React Testing Library	Unitarias y componentes (frontend)	Compatible con Vite. Se descartó Vitest 5 porque dejó de reportar cobertura de los servicios (ver nota).
SpotBugs 4.8.6 + FindSecBugs 1.13.0	Análisis estático (backend)	Detección de patrones de seguridad de Spring y Java (REQ-CAL-02).
ESLint 9 + eslint-plugin-security	Análisis estático (frontend)	Detecta inyección de objetos y expresiones regulares inseguras.
npm audit	SCA (frontend)	Vulnerabilidades conocidas en dependencias (REQ-SEG-06).
Script Python (ejecutar_casos.py)	Casos funcionales, de seguridad y de rendimiento	Reproducible, con evidencia JSON y tabla por caso.
curl y Chrome headless	Evidencias	Respuestas HTTP y capturas de Swagger y del frontend.
Docker y Docker Compose	Despliegue	Reproducibilidad del entorno (REQ-DEP-01).
Git / GitHub	Control de versiones	Trazabilidad de cambios (rama main, commit base 2d7685f).

Nota sobre Vitest 5: con @vitest/coverage-v8 5.0.4 y vite 7, las 55 pruebas pasan, pero el informe de cobertura omite los archivos de servicios (src/services), lo que reduce la cobertura medida de 68,5 % a 51,8 %. Se mantuvo la versión 1.6 para no alterar la medición de cobertura. Esta decisión deja abierto el INC-09.

3.6 Recursos (humanos, técnicos y de entorno)

Recurso	Detalle
Humanos	Tres integrantes del grupo: desarrollo y revisión de pruebas, ejecución manual y revisión de calidad. La defensa es individual.
Equipo	MacBook (arm64), macOS. Docker Desktop 29.5.2.
Software	JDK 21.0.12 (OpenJDK, Homebrew), Maven 3.9.16, Node 24.15 / npm 11.12, Python 3.14, Google Chrome.
Entorno de ejecución	Contenedores Docker: cuatro servicios Spring Boot (puertos 8080 a 8083) y frontend nginx (puerto 5173).
Entorno de línea base	Procesos locales sobre el commit 2d7685f, sin cambios, para comparar.
Datos de prueba	Catálogo semilla de 11 perfumes (data.sql). Usuarios de prueba sintéticos; no se usan datos reales de personas.

3.7 Cronograma

El cronograma asigna fechas concretas a cada fase y termina el 10 de octubre de 2026. El estado refleja la situación real del trabajo al momento de redactar el informe.

Fase	Actividad	Fechas (2026)	Estado
F0	Preparación del entorno y congelamiento de la versión (commit 2d7685f)	1 al 2 de octubre	Completada
F1	Línea base: pruebas unitarias, cobertura y ejecución de casos	3 de octubre	Completada
F2	Análisis estático (SpotBugs, ESLint) y composición de software (npm audit)	4 de octubre	Completada
F3	Mitigación de incidentes y pruebas de regresión	5 al 7 de octubre	Completada (INC-09 abierto)
F4	Despliegue en Docker y ejecución final de casos	8 de octubre	Completada
F5	Redacción del informe y verificación de cifras contra evidencias	9 de octubre	En curso
F6	Preparación de la defensa individual (guion y preguntas probables)	10 de octubre	Pendiente

4. Diseño de casos de prueba

Se diseñaron 19 casos. Cada uno incluye identificador, descripción, requisito asociado, datos de entrada, resultado esperado, criterio de aceptación y tipo de prueba. Los resultados esperados se redactaron antes de ejecutar y se ajustaron solo cuando la ejecución mostró que la especificación previa era incorrecta; en esos casos se indica en la sección 5.

ID	Descripción	Requisito	Datos de entrada	Resultado esperado	Criterio de aceptación	Tipo
CP-01	Autenticación JWT y acceso a endpoints protegidos	REQ-SEG-01	username aleatorio (usrNNN), password 'pass12345'; POST /api/auth/register y /api/auth/login; GET /api/products con y sin token	Registro y login devuelven token; GET protegido 200 con token y 401/403 sin token.	Token presente en registro y login; 200 con token; 401 o 403 sin token.	Seguridad / funcional
CP-02	Rechazo de credenciales inválidas y registro duplicado	REQ-SEG-01	username del CP-01; password incorrecta 'clave-incorrecta'; registro duplicado con la misma cuenta	Login incorrecto: HTTP 401. Registro duplicado: HTTP 409.	Códigos 401 y 409 con mensaje controlado, sin token.	Seguridad / negativo
CP-03	Descuento según tipo de orden (Factory Method)	REQ-FUNC-02	customerId 1, email test@test.com, ítem productId 1 'Sauvage' cantidad 2 precio 100; orderType WEB y MARKETPLACE	WEB: 5 %, total 190,00, STANDARD. MARKETPLACE: 10 %, total 180,00, EXPRESS.	Descuento y total exactos; método de envío correspondiente.	Funcional / caja negra
CP-04	Rechazo de tipo de orden no soportado	REQ-FUNC-02	orderType 'INVALIDO' con el mismo ítem que CP-03	Rechazo controlado con HTTP 400 y mensaje 'Tipo de orden no soportado'.	HTTP 400; no se crea el pedido.	Funcional / negativo
CP-05	Fallback del Circuit Breaker ante fallo del carrier	REQ-RES-01	orderId 999, address 'Calle 123', city 'Santiago', country 'Chile'; carrier 'DHL' y 'FAIL_CARRIER'	DHL: TRK-, sin fallback, entrega +3 días. FAIL_CARRIER: FBK-, fallback=true, entrega +7 días, HTTP 201.	Ambos casos devuelven 201; el fallo no provoca error 5xx.	Resiliencia
CP-06	Búsqueda y filtrado de productos	REQ-FUNC-03	Filtros: brand=Dior, fragrance=Amaderado, price-range 70000-110000, in-stock, search=SAUVAGE	Listado completo de 11 productos; cada filtro devuelve solo coincidencias.	Todos los elementos cumplen el criterio; búsqueda por nombre insensible a mayúsculas.	Funcional / caja negra
CP-07	Validación de entradas en servicios de negocio	REQ-VAL-01	Producto name '' price 100 y price -50; pedido email 'invalido' items []; envío sin carrier	HTTP 400 en cada caso, y ningún registro persistido.	Todos los datos inválidos rechazados con 400; el conteo de productos no cambia.	Seguridad / validación
CP-08	Validación de entradas a través del gateway (BFF)	REQ-VAL-01	Producto name '' enviado a través del BFF con token válido	HTTP 400 propagado desde el servicio de inventario.	El código de error de validación llega íntegro al cliente.	Seguridad / integración
CP-09	Tiempos de respuesta (lectura y escritura)	REQ-NFR-01	20 GET /api/products vía BFF y 20 POST /api/orders (WEB, ítem 1 x 1); medición en cliente	p95 lectura < 500 ms; p95 escritura < 1000 ms.	Percentil 95 dentro del límite; medición en localhost, sin carga concurrente.	No funcional / rendimiento
CP-10	Protección de contraseñas en	REQ-SEG-02	Archivo AuthService.java (inspección	Las contraseñas se guardan como hash (p. ej. BCrypt) y nunca en	No existe asignación de la contraseña en claro; se usa	Seguridad / inspección de

	almacenamiento		estática)	texto plano.	un PasswordEncoder.	código
CP-11	Errores sin exposición de detalles internos	REQ-SEG-03	GET /api/orders/abc (id no numérico)	Respuesta de error genérica, sin excepciones ni mensajes del framework.	El cuerpo no contiene texto técnico (p. ej. 'Failed to convert', 'For input string', 'java.', 'org.springframework').	Seguridad / manejo de errores
CP-12	Restricción de orígenes CORS	REQ-SEG-04	OPTIONS /api/products con Origin http://evil.example y http://localhost:5173	Origen no autorizado sin Access-Control-Allow-Origin; origen localhost:5173 autorizado.	Solo los orígenes definidos reciben la cabecera de permiso.	Seguridad / configuración
CP-13	Secreto de firma JWT fuera del código fuente	REQ-SEG-05	bff-gateway/src/main/resources/application.yml (inspección estática)	jwt.secret se resuelve desde una variable de entorno (sin valor literal).	El valor del secreto no aparece literal en el repositorio.	Seguridad / configuración
CP-14	Cobertura de líneas JaCoCo >= 60 %	REQ-CAL-01	target/site/jacoco/jacoco.csv de cada servicio tras mvn verify	Cobertura de líneas >= 60 % por servicio.	Cada módulo supera el umbral; el gate de JaCoCo está activo.	No funcional / mantenibilidad
CP-15	Sin hallazgos de prioridad 1 en análisis estático (SpotBugs + FindSecBugs)	REQ-CAL-02	target/spotbugsXml.xml de cada servicio tras análisis SpotBugs (perfil security)	0 hallazgos de prioridad 1.	Ningún hallazgo de prioridad 1 sin tratamiento documentado.	Análisis estático
CP-16	Dependencias del frontend sin vulnerabilidades críticas o altas	REQ-SEG-06	npm audit --json sobre frontend/package-lock.json	0 vulnerabilidades críticas o altas.	Ninguna vulnerabilidad crítica o alta en dependencias de producción ni de desarrollo.	Seguridad / SCA
CP-17	Despliegue reproducible en contenedores Docker	REQ-DEP-01	Dockerfile por servicio y docker-compose.yml (docker compose config)	Dockerfile en cada servicio y docker-compose.yml válido.	docker compose config sin errores.	Despliegue / operación
CP-18	Aislamiento de los microservicios de negocio	REQ-SEG-07	GET sin token a http://localhost:8081/api/products, :8082/api/orders/customer/1 y :8083/api/shipments	Ninguna petición sin token devuelve datos: el acceso debe ser rechazado o no alcanzable desde fuera de la red interna.	Respuesta distinta de 200 o puerto no accesible desde el host.	Seguridad / control de acceso
CP-19	Alta de producto con persistencia y asignación de ID	REQ-FUNC-04	name 'Producto CP19 <marca temporal>', brand 'Prueba', price 10.5, stockQuantity 5, fragrance 'Test'; token del CP-01	HTTP 201 con id asignado; el producto aparece en el catálogo.	Código 201, id no nulo y producto recuperable por GET.	Funcional / caja negra

4.1 Matriz de trazabilidad requisito → caso → incidente

Requisito	Descripción	Origen	Casos	Base	Final	Incidentes
REQ-SEG-01	Autenticación con JWT; rechazo de accesos sin token o con credenciales inválidas.	Ev1 REQ-SEG-01; Ev1 §2.2 (control de acceso)	CP-01, CP-02	FALLA	PASA	INC-04, INC-12, INC-13
REQ-FUNC-02	Descuento WEB 5 %, MARKETPLACE 10 %, envío STANDARD/EXPRESS; rechazo de tipo	Ev1 REQ-FUNC-02	CP-03, CP-04	FALLA	PASA	INC-05

	de orden no soportado.					
REQ-FUNC-03	Búsqueda por marca, fragancia, rango de precio, stock y nombre.	Ev1 REQ-FUNC-03	CP-06	FALLA	PASA	INC-17
REQ-RES-01	Fallback automático del Circuit Breaker ante fallo del carrier externo.	Ev1 REQ-RES-01; Ev1 §2.1 (tolerancia a fallos)	CP-05	PASA	PASA	-
REQ-VAL-01	Validación de datos de entrada con rechazo HTTP 400 y sin persistencia.	Ev1 REQ-VAL-01; Ev1 §2.2 (integridad)	CP-07, CP-08	FALLA	PASA	INC-06
REQ-NFR-01	Tiempos de respuesta: lectura < 500 ms y escritura < 1000 ms (p95).	Ev1 §2.1 (rendimiento)	CP-09	PASA	PASA	-
REQ-SEG-02	Las contraseñas se almacenan como hash, nunca en claro.	Ev1 §2.2 (protección de datos); OWASP A02	CP-10	FALLA	PASA	INC-01
REQ-SEG-03	Los errores no exponen detalles internos al cliente.	Ev1 §2.2 (manejo de errores); OWASP A05	CP-11	FALLA	PASA	INC-03
REQ-SEG-04	CORS restringido a los orígenes definidos.	Ev1 §2.2 (CORS)	CP-12	PASA	PASA	-
REQ-SEG-05	El secreto de firma JWT no figura en el código fuente.	Hallazgo de revisión de código; OWASP A02 y A05	CP-13	FALLA	PASA	INC-02
REQ-SEG-06	Sin vulnerabilidades críticas o altas en dependencias del frontend.	Análisis de composición de software (npm audit); OWASP A06	CP-16	FALLA	FALLA	INC-09
REQ-CAL-01	Cobertura de líneas JaCoCo >= 60 % en cada servicio backend.	Ev1 §2.1 (mantenibilidad)	CP-14	FALLA	PASA	INC-07, INC-11, INC-14
REQ-CAL-02	Sin hallazgos de prioridad 1 en el análisis estático (SpotBugs + FindSecBugs).	Pauta EP3, IL 2.3; criterio de aceptación propio	CP-15	FALLA	PASA	INC-08
REQ-SEG-07	Los microservicios de negocio solo son accesibles a través del BFF autenticado.	Ev1 §2.2 (control de acceso); hallazgo de revisión de arquitectura; OWASP A01	CP-18	FALLA	PASA	INC-15
REQ-FUNC-04	Alta de producto con persistencia y asignación de identificador.	Ev1 §3.2 (pruebas funcionales de CRUD de productos, prioridad alta)	CP-19	FALLA	PASA	INC-16
REQ-DEP-01	Despliegue reproducible en contenedores Docker.	Pauta EP3 (imágenes Docker del software)	CP-17	FALLA	PASA	INC-10

Cobertura del conjunto declarado: 16 de 16 requisitos tienen al menos un caso asociado (100 %). Esta cobertura se refiere al conjunto de la sección 1.3 y no a un ERS oficial. Requisitos sin prueba automatizada: usabilidad y compatibilidad de navegadores (ver sección 3.3).

5. Ejecución de pruebas

5.1 Resumen de ejecución

Casos diseñados: 19. Casos ejecutados: 19 sobre la línea base y 19 sobre la versión final, en ambos casos con el mismo ejecutor. Resultados: línea base, 5 aprobados y 14 fallidos; versión final, 18 aprobados y 1 fallido. Ningún caso quedó bloqueado ni sin ejecutar. El caso fallido de la versión final (CP-16) corresponde a un riesgo residual documentado en INC-09.

Indicador	Línea base (2d7685f)	Versión final
Casos aprobados / fallidos	5 / 14	18 / 1
Pruebas JUnit (backend)	86	121 (0 fallos)
Pruebas Vitest (frontend)	55 (0 fallos)	55 (0 fallos)
Cobertura de líneas: inventario	83.7 %	95.2 %
Cobertura de líneas: pedidos	83.8 %	88.7 %
Cobertura de líneas: envíos	79.8 %	86.3 %
Cobertura de líneas: BFF	71.1 %	74.3 %
SpotBugs prioridad 1 (backend)	1 (CSRF)	0 (1 riesgo aceptado documentado)
npm audit: críticas / altas	3 / 13	3 / 6
ESLint (frontend, seguridad)	no ejecutado	58 avisos (47 no-unused-vars, 8 object-injection, 2 hooks, 1 regex)
Despliegue Docker	no existía	4 servicios + frontend levantados

Nota de medición. En el commit original no se generaba el reporte JaCoCo (INC-07); la cobertura de la línea base se midió sobre el mismo código con el argLine corregido. Los valores finales se leen de target/site/jacoco de cada servicio. La línea base de la batería de casos usa el mismo ejecutor que la versión final (19 casos).

5.2 Detalle de ejecución de casos

Caso	Descripción	Base	Final	Fecha	Responsable	Evidencia
CP-01	Autenticación JWT y acceso a endpoints protegidos	PASA	PASA	2026-10-14	Equipo (script)	E04 / E05 / resultados.json

CP-02	Rechazo de credenciales inválidas y registro duplicado	FALLA	PASA	2026-10-14	Equipo (script)	E04 / E05 / resultados.json
CP-03	Descuento según tipo de orden (Factory Method)	PASA	PASA	2026-10-14	Equipo (script)	E04 / E05 / resultados.json
CP-04	Rechazo de tipo de orden no soportado	FALLA	PASA	2026-10-14	Equipo (script)	E04 / E05 / resultados.json
CP-05	Fallback del Circuit Breaker ante fallo del carrier	PASA	PASA	2026-10-14	Equipo (script)	E04 / E05 / resultados.json
CP-06	Búsqueda y filtrado de productos	FALLA	PASA	2026-10-14	Equipo (script)	E04 / E05 / resultados.json
CP-07	Validación de entradas en servicios de negocio	FALLA	PASA	2026-10-14	Equipo (script)	E04 / E05 / resultados.json
CP-08	Validación de entradas a través del gateway (BFF)	FALLA	PASA	2026-10-14	Equipo (script)	E04 / E05 / resultados.json
CP-09	Tiempos de respuesta (lectura y escritura)	PASA	PASA	2026-10-14	Equipo (script)	E04 / E05 / resultados.json
CP-10	Protección de contraseñas en almacenamiento	FALLA	PASA	2026-10-14	Equipo (script)	E04 / E05 / resultados.json
CP-11	Errores sin exposición de detalles internos	FALLA	PASA	2026-10-14	Equipo (script)	E04 / E05 / resultados.json
CP-12	Restricción de orígenes CORS	PASA	PASA	2026-10-14	Equipo (script)	E04 / E05 / resultados.json
CP-13	Secreto de firma JWT fuera del código fuente	FALLA	PASA	2026-10-14	Equipo (script)	E04 / E05 / resultados.json
CP-14	Cobertura de líneas JaCoCo >= 60 %	FALLA	PASA	2026-10-14	Equipo (script)	E04 / E05 / resultados.json
CP-15	Sin hallazgos de prioridad 1 en análisis estático (SpotBugs + FindSecBugs)	FALLA	PASA	2026-10-14	Equipo (script)	E04 / E05 / resultados.json
CP-16	Dependencias del frontend sin vulnerabilidades críticas o altas	FALLA	FALLA	2026-10-14	Equipo (script)	E04 / E05 / resultados.json
CP-17	Despliegue reproducible en contenedores Docker	FALLA	PASA	2026-10-14	Equipo (script)	E04 / E05 / resultados.json
CP-18	Aislamiento de los microservicios de negocio	FALLA	PASA	2026-10-14	Equipo (script)	E04 / E05 / resultados.json
CP-19	Alta de producto con persistencia y asignación de ID	FALLA	PASA	2026-10-14	Equipo (script)	E04 / E05 / resultados.json

Detalle de los casos con diferencia entre línea base y versión final:

- **CP-02 (FALLA → PASA):** Login con contraseña incorrecta [POST http://localhost:8080/api/auth/login] -> HTTP 403; Registro de usuario ya existente [POST http://localhost:8080/api/auth/register] -> HTTP 403
- **CP-04 (FALLA → PASA):** Pedido con tipo INVALIDO [POST http://localhost:8080/api/orders] -> HTTP 403
- **CP-06 (FALLA → PASA):** Filtro brand/Dior: 0 resultados, todos coinciden; Filtro fragrance/Amaderado: 0 resultados, todos coinciden
- **CP-07 (FALLA → PASA):** Producto con nombre vacío [POST http://localhost:8080/api/products] -> HTTP 403; Producto con precio negativo [POST http://localhost:8080/api/products] -> HTTP 403
- **CP-08 (FALLA → PASA):** Producto con nombre vacío vía BFF [POST http://localhost:8080/api/products] -> HTTP 403
- **CP-10 (FALLA → PASA):** No se almacena request.getPassword() en claro; Se utiliza un PasswordEncoder (hash)
- **CP-11 (FALLA → PASA):** Verificación de ausencia de detalle técnico en el cuerpo de error (HTTP 404)
- **CP-13 (FALLA → PASA):** jwt.secret usa placeholder ${...}
- **CP-14 (FALLA → PASA):** inventory-service: reporte JaCoCo existe; order-service: reporte JaCoCo existe
- **CP-15 (FALLA → PASA):** bff-gateway: hallazgos prioridad 1 = 1 ['SPRING_CSRF_PROTECTION_DISABLED']
- **CP-17 (FALLA → PASA):** inventory-service/Dockerfile presente; order-service/Dockerfile presente
- **CP-18 (FALLA → PASA):** Acceso directo sin token a inventario [GET http://localhost:8081/api/products] -> HTTP 200; Acceso directo sin token a pedidos (datos de clientes) [GET http://localhost:8082/api/orders/customer/1] -> HTTP 200
- **CP-19 (FALLA → PASA):** POST /api/products vía BFF -> HTTP 403; El producto recibe un id asignado

5.2.1 Evidencias visuales

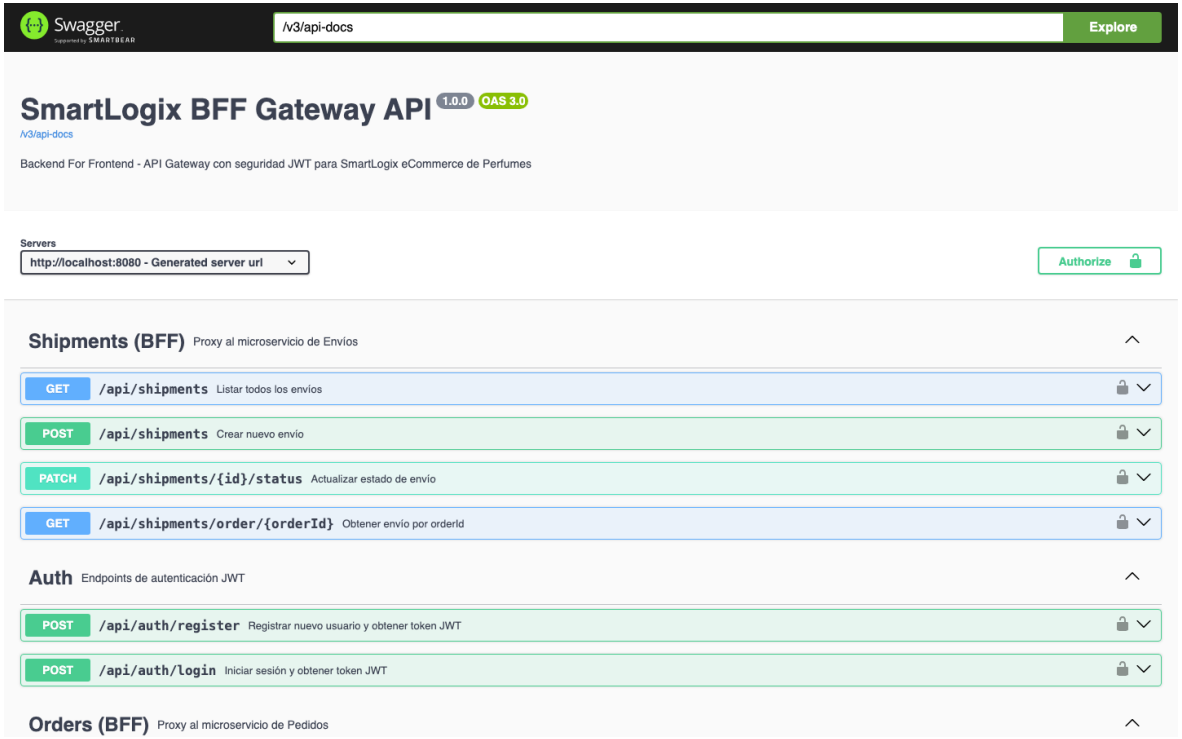


Figura 1. Swagger UI del BFF Gateway (documentación OpenAPI 3.0 generada por springdoc).

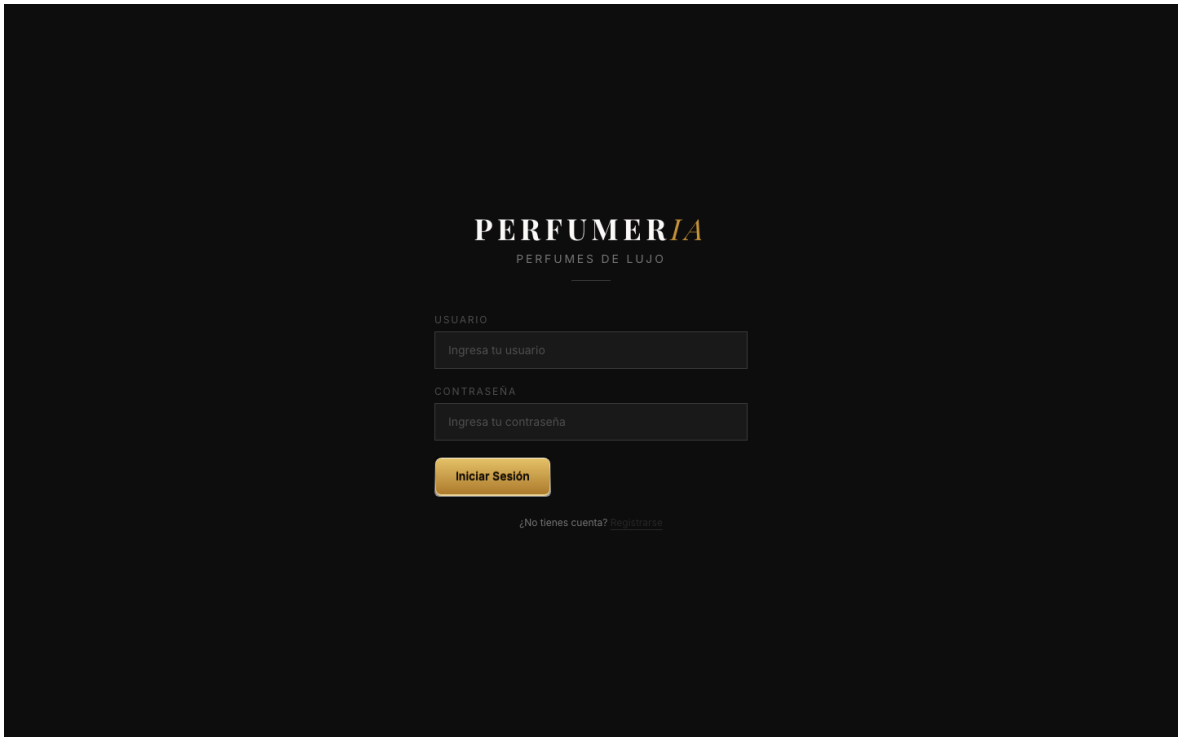


Figura 2. Pantalla de inicio de sesión del frontend desplegado en Docker (puerto 5173). Ver INC-14.

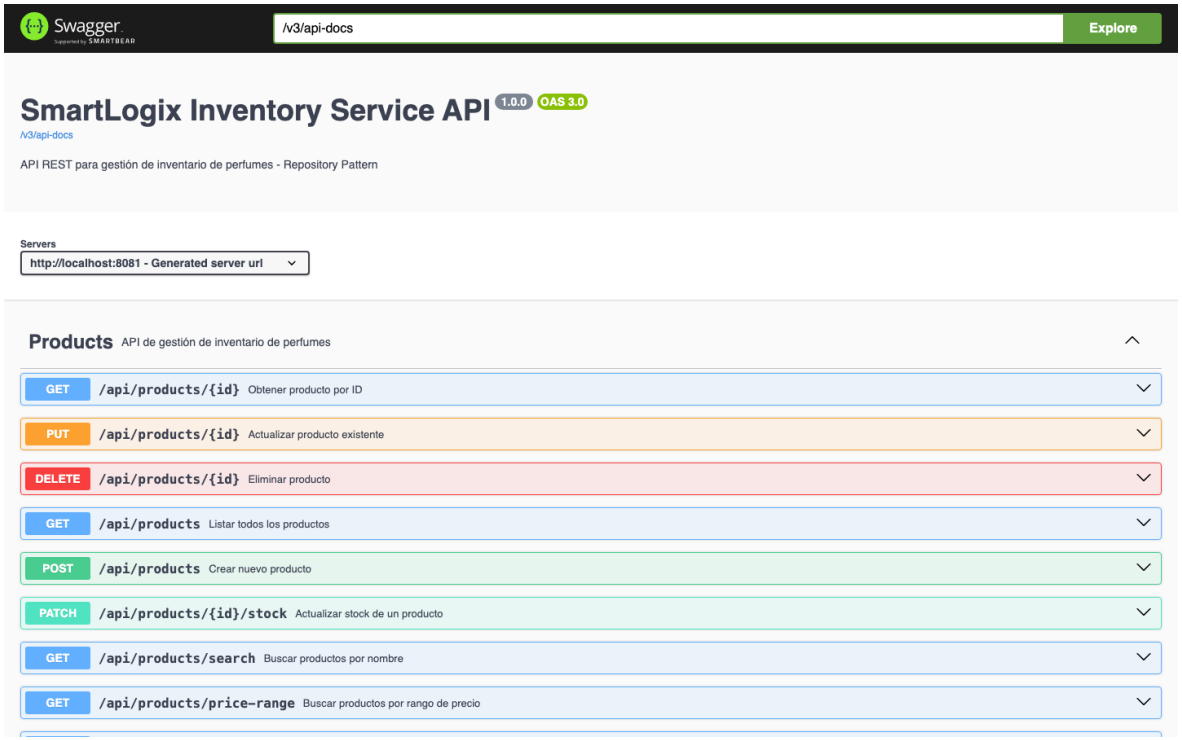


Figura 3. Swagger UI del Inventory Service (puerto 8081).

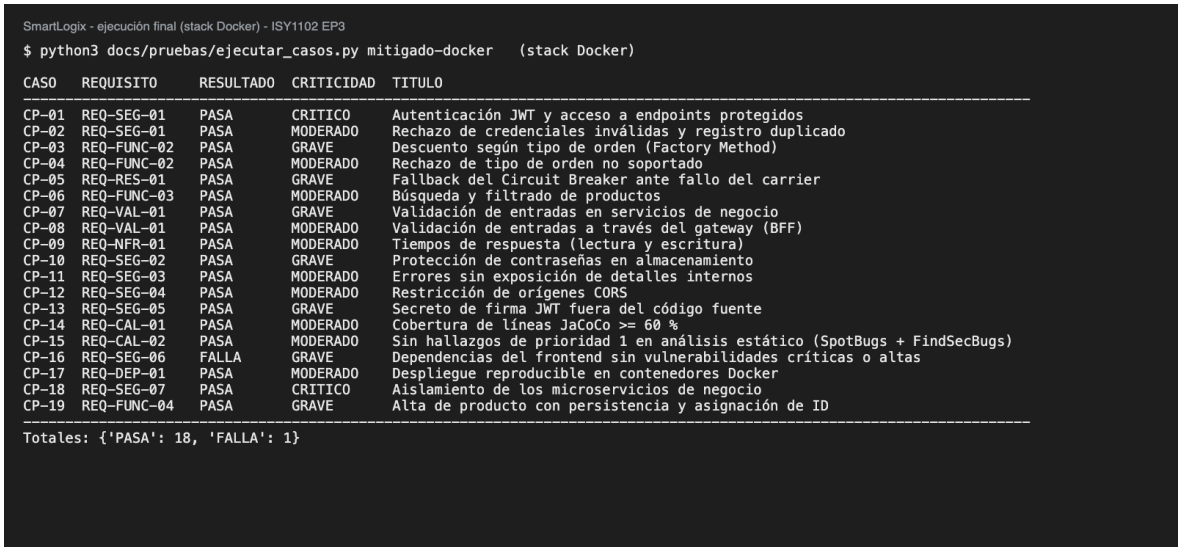


Figura 4. Resultados de la ejecución final de los 19 casos (docs/evidencias/mitigado-docker/resultados.md).

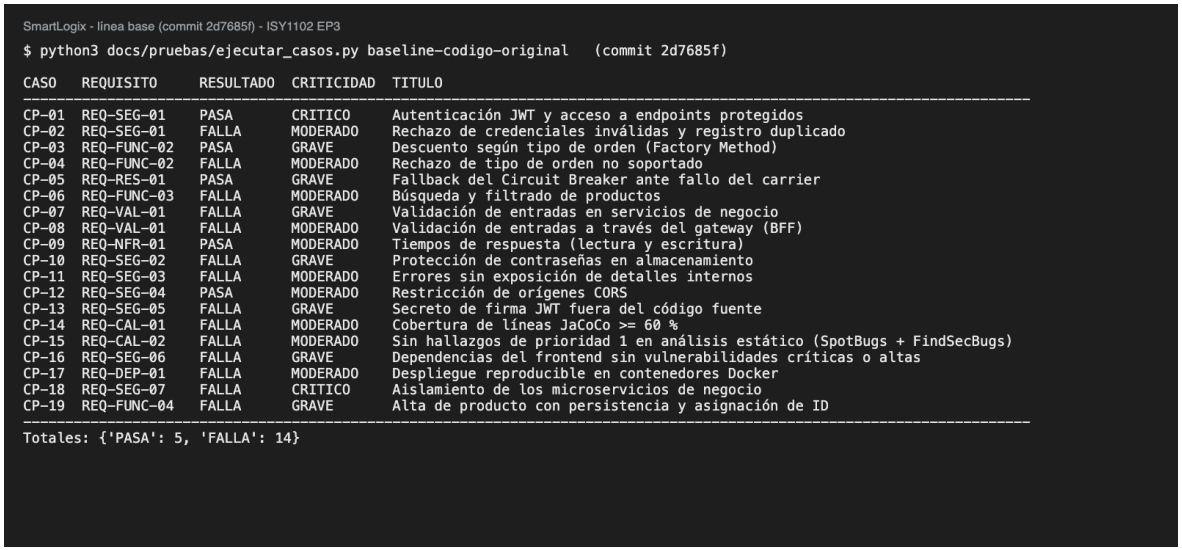


Figura 5. Resultados de la línea base sobre el código original (commit 2d7685f).

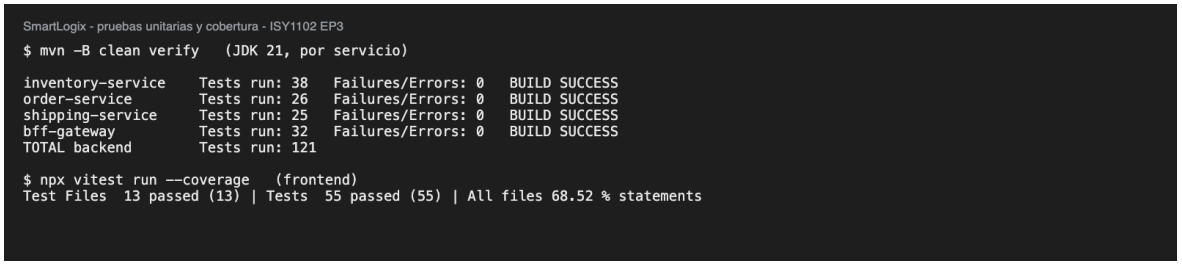


Figura 6. Pruebas unitarias y cobertura por servicio.

Evidencia textual (respuestas HTTP reales) en docs/evidencias/capturas/E02-curl-seguridad-validacion.txt: sin token, 401; con token válido, 200; tipo de orden inválido, 400 con mensaje controlado; producto con nombre vacío, 400 indicando el campo.

5.3 Incidentes identificados y criticidad

Escala de criticidad, según la plantilla de la asignatura: CRÍTICO (impide el funcionamiento o compromete credenciales o datos; requiere remediación inmediata), GRAVE (afecta funcionalidades importantes o la cadena de desarrollo; debe corregirse con urgencia), MODERADO (afecta casos específicos; se corrige en la siguiente iteración) y LEVE (apariencia o texto).

ID	Caso / origen	Requisito	Descripción	Evidencia (ubicación)	Criticidad	Impacto	Estado	Tratamiento
INC-01	CP-10	REQ-SEG-02	Contraseñas de usuario almacenadas en texto plano en un mapa en memoria y comparadas con igualdad directa.	bff-gateway/.../AuthService.java (versión base: líneas 17, 24, 39)	CRÍTICO	Compromiso de credenciales de todos los usuarios ante cualquier acceso a memoria o volcado.	CERRADO	Hash BCrypt (PasswordEncoder) y prueba register_shouldStoreBcryptHashNotPlaintext.
INC-02	CP-13	REQ-SEG-05	Secreto de firma JWT escrito literalmente en application.yml.	bff-gateway/src/main/resources/application.yml (versión base: línea 17)	CRÍTICO	Quien conozca el secreto puede firmar tokens válidos para cualquier usuario.	CERRADO (con reserva)	Secreto desde ${JWT_SECRET}, sin valor por defecto. El valor sigue en el historial de git: debe considerarse expuesto y rotarse.
INC-03	CP-11	REQ-SEG-03	Excepciones de conversión devuelven HTTP 404 con el mensaje interno del framework.	order-service y demás servicios: GlobalExceptionHandler (versión base)	MODERADO	Revela tecnología y estructura interna; el código de estado es incorrecto.	CERRADO	Handler que responde 400/404/500 con mensaje genérico; errores no controlados se registran en log.
INC-04	CP-02	REQ-SEG-01	Credenciales inválidas y registro duplicado responden HTTP 403 sin código controlado.	bff-gateway (versión base: sin GlobalExceptionHandler)	MODERADO	Código de estado incorrecto para el cliente y para la auditoría de accesos.	CERRADO	401 para credenciales inválidas (mismo mensaje para usuario inexistente o clave errónea) y 409 para duplicados.
INC-05	CP-04	REQ-FUNC-	Tipo de orden no soportado responde HTTP 404 en lugar de	order-service/.../OrderService.java	MODERADO	El cliente interpreta el error como recurso	CERRADO	IllegalArgumentException

		02	400.	(versión base: RuntimeException)		inexistente.		mapeada a 400.
INC-06	CP-08	REQ-VAL-01	Un error 400 de validación del servicio de inventario llega al cliente a través del BFF como HTTP 403.	bff-gateway (versión base). Causa probable: excepción no tratada y despacho de error protegido por autenticación	MODERADO	El cliente no recibe el motivo del rechazo.	CERRADO	Propagación de errores 4xx del servicio de destino y ruta /error pública. La causa es probable (no se confirmó con logs del BFF); el cambio elimina el síntoma observado.
INC-07	CP-14	REQ-CAL-01	El gate de cobertura JaCoCo no se ejecutaba: el argLine de Surefire sustituía al agente y no se generaba jacoco.exec.	pom.xml de los cuatro servicios (versión base: <argLine>)	MODERADO	La cobertura declarada no se verificaba en el build; el umbral era ineficaz.	CERRADO	argLine = @{argLine} -Dnet.bytebuddy.experimental=true. Cobertura medida en cada servicio.
INC-08	CP-15	REQ-CAL-02	SpotBugs de prioridad 1: SPRING_CSRF_PROTECTION_DISABLED en SecurityConfig.	bff-gateway/.../SecurityConfig.java:28	MODERADO	Riesgo CSRF si el token se transportara en cookies.	RIESGO ACEPTADO	Filtro de exclusión documentado (bff-gateway/spotbugs-exclude.xml). La API usa Bearer y sesión STATELESS. Se oculta el hallazgo del reporte; se declara aquí para transparencia.
INC-09	CP-16	REQ-SEG-06	Vulnerabilidades en dependencias de desarrollo: 3 críticas (vitest, @vitest/coverage-v8, tinypool) y 6 altas (vite, tailwindcss, braces, chokidar, fast-glob, micromatch).	frontend/package.json y package-lock.json	GRAVE	Impacto en la cadena de build y de pruebas; no en las dependencias de ejecución (axios quedó corregido).	ABIERTO	Corregir exige saltos de versión mayor (vitest 5, vite 7 o 8, tailwind 4). Vitest 5 se probó y redujo la cobertura medida de 68,5 % a 51,8 %, por lo que se descartó (ver sección 3.5).
INC-10	CP-17	REQ-DEP-01	El repositorio no contenía Dockerfile ni docker-compose.yml.	Raíz del repositorio (versión base)	MODERADO	El despliegue no es reproducible y no cumple el insumo 'imágenes Docker' de la pauta.	CERRADO	Dockerfile multi-etapa por servicio, nginx como proxy de /api y docker-compose.yml con JWT_SECRET obligatorio.
INC-11	Revisión documental	REQ-CAL-01	Cifras de cobertura de la Ev1 (87,4 % backend y 84,1 % frontend) no reproducibles.	docs de la Ev1 frente a frontend/coverage y target/site/jacoco	MODERADO	Afirmaciones no verificables en la evaluación anterior.	CERRADO	Cifras corregidas en este informe con los valores medidos (sección 5.1).
INC-12	Revisión de código	REQ-SEG-01	El token JWT se guarda en localStorage, accesible desde cualquier script de la página.	frontend/src/context/AuthContext.jsx:30	MODERADO	Un fallo de XSS permitiría robar la sesión.	ABIERTO	No se encontraron sinks de XSS (dangerouslySetInnerHTML, innerHTML, eval). Mitigación recomendada: cookie HttpOnly con SameSite y CSRF habilitado.
INC-13	Revisión de código	REQ-SEG-01	El endpoint de login no limita los intentos de autenticación (no existe rate limiting en el BFF).	bff-gateway/.../AuthController.java	MODERADO	Permite ataques de fuerza bruta sobre contraseñas.	ABIERTO	Recomendado: límite por IP y usuario (p. ej. Bucket4j o Resilience4j RateLimiter) y bloqueo temporal.
INC-15	CP-18	REQ-SEG-07	Los microservicios de negocio responden sin token en sus puertos: inventario (catálogo), pedidos (datos de clientes con nombre y correo) y	docker-compose.yml (versión base: puertos 8081-8083 publicados) y ausencia de Spring Security en	CRÍTICO	Cualquier usuario de la red puede leer pedidos con datos personales y crear productos sin	MITIGADO (parcial)	Puertos 8081-8083 dejan de publicarse; el acceso solo es posible dentro de la red de compose. Pendiente: que cada

			envíos.	inventory, order y shipping		pasar por el BFF; el control de acceso del gateway se elude.		microservicio valide el JWT (defensa en profundidad).
INC-16	CP-19 / CP-07	REQ-FUNC-04	El alta de productos falla siempre con HTTP 500: la secuencia de IDs no avanza después de la semilla con IDs explícitos y el INSERT viola la clave primaria.	inventory-service/src/main/resources/data.sql (versión base) y ProductService.createProduct	GRAVE	No es posible incorporar productos al catálogo; función de administración prioritaria.	CERRADO	ALTER TABLE product ALTER COLUMN id RESTART WITH 100. Regresión: ProductSeedIdIntegrationTest falla sin la corrección y pasa con ella.
INC-17	CP-06	REQ-FUNC-03	El BFF no expone los filtros de búsqueda del inventario (marca, fragancia, precio, stock y nombre). Para el cliente, la búsqueda no existe.	bff-gateway/.../controller/ProductController.java (versión base: solo listado, detalle, alta, actualización y stock)	MODERADO	La búsqueda solo es accesible si se llama directamente al servicio, lo que el control de acceso ya no permite.	CERRADO	Endpoints de búsqueda en el BFF con codificación segura de segmentos y parámetros (InventoryProxyServiceTest, 3 pruebas).
INC-14	Observación visual	REQ-CAL-01	La pantalla de inicio se presenta como 'PERFUMERÍA', mientras el título HTML es 'SmartLogix'.	Captura E07 (docs/evidencias/capturas)	LEVE	Inconsistencia de marca; no afecta la funcionalidad.	ABIERTO	Unificar el nombre comercial.

Resumen: 17 incidentes (CRÍTICO: 3, MODERADO: 11, GRAVE: 2, LEVE: 1). Abiertos o con riesgo aceptado: INC-08, INC-09, INC-12, INC-13, INC-14. Los incidentes CRÍTICOS INC-01 e INC-02 están cerrados; INC-02 requiere además rotar el secreto, porque permanece en el historial del repositorio. INC-15 (CRÍTICO) está mitigado solo en parte: los puertos ya no se publican, pero los microservicios no validan el token por sí mismos.

6. Análisis ético y normativo de los resultados

6.1 Análisis ético

- **Independencia de la evaluación:** el equipo evalúa código propio y redactó los resultados esperados. Para reducir el sesgo, los resultados esperados se fijaron antes de ejecutar y las discrepancias se documentan como incidentes en lugar de corregir la expectativa en silencio.
- **Divulgación responsable:** las vulnerabilidades se describen con ubicación exacta para permitir su corrección, pero el informe no reproduce el valor del secreto ni contraseñas reales. El valor literal del secreto se registró como 'oculto' en la evidencia.
- **Alcance autorizado:** todas las pruebas se ejecutaron sobre entornos locales y contenedores propios, sin apuntar a sistemas de terceros, en línea con las instrucciones de la asignatura.
- **Honestidad en la medición:** los cambios de cobertura que resultaban incómodos (Vitest 5) se reportan igual que los favorables. Un riesgo aceptado (CSRF en INC-08) también se declara, porque ocultarlo en el reporte sería engañoso.
- **Responsabilidad sobre los datos:** el sistema maneja datos personales; un fallo de confidencialidad afecta a personas que no participaron en el desarrollo. Esto justifica priorizar INC-01 y INC-02.

6.2 Análisis normativo

- **Seguridad de los datos (art. 14 quinquies de la Ley 21.719):** las contraseñas en texto plano (INC-01) incumplen el deber de adoptar medidas que aseguren la confidencialidad. La corrección con BCrypt es una medida adecuada, aunque no suficiente por sí sola: falta el cifrado de la base de datos en reposo, que hoy es H2 en memoria.
- **Reporte de vulneraciones (art. 14 sexies):** si un incidente como INC-01 o INC-02 se hubiera materializado sobre datos reales, el responsable debería reportarlo a la Agencia sin dilaciones indebidas. El sistema actual no cuenta con un procedimiento de respuesta a incidentes ni con registro de vulneraciones.
- **Derechos del titular (arts. 4, 9 y 10):** no se identificaron endpoints para consultar, rectificar, suprimir u obtener la portabilidad de los datos de un usuario. Se verificó por búsqueda en el código. Es un incumplimiento que debe corregirse antes de un uso real.
- **Consentimiento y transparencia:** no se identificó en la interfaz un mecanismo de consentimiento para el tratamiento de datos al registrarse. Esto se verificó en el formulario de registro y no fue probado de forma exhaustiva en el navegador.
- **Límite de intentos y autenticación robusta (OWASP A07):** la ausencia de límites de intentos (INC-13) y el almacenamiento del token en localStorage (INC-12) están fuera de lo que recomiendan OWASP y las buenas prácticas de gestión de sesiones.

- **Dependencias (OWASP A06):** las vulnerabilidades críticas en herramientas de desarrollo (INC-09) no afectan directamente a los usuarios finales, pero sí a la integridad del proceso de construcción. El riesgo residual debe comunicarse.

Conclusión normativa: el sistema avanza hacia el cumplimiento de la seguridad técnica, pero no cumple todavía con las obligaciones de derechos del titular, consentimiento ni respuesta a incidentes. Estas brechas se deben cerrar antes de tratar datos reales de personas.

7. Conclusiones y recomendaciones

La línea base cumplió 5 de 19 casos; tras las mitigaciones, la versión final cumple 18 de 19. Las mitigaciones cerraron los incidentes críticos INC-01 (credenciales en claro) e INC-02 (secreto JWT literal), y corrigieron el manejo de errores, la validación del gateway, el gate de cobertura, el alta de productos y el despliegue. El acceso directo a los microservicios (INC-15, crítico) se mitigó a nivel de red, pero no a nivel de aplicación. Los incidentes abiertos restantes son INC-09 (GRAVE), INC-12, INC-13 (MODERADO) e INC-14 (LEVE).

- R1. Rotar el secreto JWT y reescribir el historial de git si el repositorio es público.
- R2. Actualizar la cadena de herramientas del frontend (vitest, vite, tailwind) y revalidar la cobertura antes de declarar cerrado INC-09.
- R3. Implementar límite de intentos en /api/auth/login (INC-13).
- R4. Migrar el token a cookie HttpOnly con SameSite y reactivar la protección CSRF (INC-08, INC-12).
- R5. Implementar los endpoints de derechos del titular y un registro de consentimiento (sección 6.2).
- R6. Cifrar los datos personales en reposo y reemplazar H2 en memoria por una base persistente antes de un uso real.
- R0. Validar el JWT en cada microservicio (filtro compartido) para completar la defensa en profundidad de INC-15.
- R7. Incorporar las pruebas de este plan a un pipeline de integración continua (GitHub Actions) que ejecute el script de casos y el análisis estático en cada cambio.

Anexo A. Reproducibilidad

Comandos para repetir la evaluación:

```
cp .env.example .env   # definir JWT_SECRET
docker compose up -d --build
python3 docs/pruebas/ejecutar_casos.py <etiqueta>
cd <servicio> && mvn -B clean verify   # pruebas y JaCoCo
cd <servicio> && mvn -Psecurity com.github.spotbugs:spotbugs-maven-plugin:4.8.6.6:spotbugs
cd frontend && npx vitest run --coverage && npm audit
```

Anexo B. Cambios realizados al código

- bff-gateway: AuthService con BCrypt y excepciones tipadas; GlobalExceptionHandler; SecurityConfig con PasswordEncoder y entry point 401; jwt.secret desde variable de entorno; endpoints de búsqueda en ProductController e InventoryProxyService. Pruebas: 32.
- inventory-service, order-service, shipping-service: ResourceNotFoundException, GlobalExceptionHandler sin mensajes internos, IllegalArgumentException para tipo de orden; corrección de la secuencia de IDs en data.sql. Pruebas: 38, 26 y 25.
- Los cuatro pom.xml: argLine corregido, perfil security con SpotBugs y FindSecBugs, filtro de exclusión documentado en el BFF.
- frontend: ESLint 9 con reglas de seguridad, npm audit fix sin cambios de versión mayor, Dockerfile y nginx.conf.
- Raíz: docker-compose.yml (sin puertos publicados para los microservicios), .env.example, .gitignore (.env), docs/pruebas/ejecutar_casos.py, docs/informe/generar_informe.py.